AADK 11: Task 3 — UI Item Interaction Design & Stateful Favourite Feature

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – **Session 11**

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Favourite Feature Design

Each list item will include a **heart icon** that allows the user to mark a destination as a favourite.

Behaviour

- When the user taps the **heart icon**, the isFavourite state toggles.
- If isFavourite = true
    - The icon changes to a **filled heart**.
    - The icon color becomes **red**.
- If isFavourite = false
    - The icon shows an **outlined heart**.

Optional Feature

Favourite items could appear in a **separate "Favourites" section** at the top of the list.

Example UI Concept

2. State Management Plan

To manage favourite states properly in Jetpack Compose, the state should **not be stored only inside the list item** because LazyColumn reuses items during scrolling.

Instead, the **state will be stored in the main list data model or ViewModel**.

State Variables

Example state list:

var destinations by mutableStateOf(listOf<TravelDestination>())

Each item contains:

isFavourite: Boolean

Toggle Logic

When the user taps the icon:

```
fun toggleFavourite(destination: TravelDestination) {
   destinations = destinations.map {
      if (it.title == destination.title)
         it.copy(isFavourite = !it.isFavourite)
      else it
   }
}
```

Why This Approach

- State remains consistent across **recomposition**
- Favourite status **does not reset during scrolling**
- Centralized state management makes UI updates easier

Using a **ViewModel** is recommended for larger apps.

---

3. Item Expansion Plan

Each item can expand to display **more information about the destination**.

Interaction Behaviour

When the user taps anywhere on the card:

- The item expands
- The **full description becomes visible**
- The image may become larger

When tapped again:

- The item collapses

UI Expansion Concept

Collapsed state:

```
---------------------------------
| Image | Title                 |
|       | Country               |
---------------------------------
```

Expanded state:

```
---------------------------------
| Image (larger)                |
| Title                         |
| Country                       |
| Full description text...      |
---------------------------------
```

Compose Implementation Strategy

Example state:

var expanded by remember { mutableStateOf(false) }

Expansion logic:

Card(

```
    modifier = Modifier
        .fillMaxWidth()
        .clickable { expanded = !expanded }
        .animateContentSize()
) {
    Column {
        Text(destination.title)

        if (expanded) {
            Text(destination.description)
        }
    }
}
```

Compose Techniques Used

Feature	Purpose
remember	Stores UI state
mutableStateOf()	Allows recomposition
animateContentSize()	Smooth expansion animation
if(expanded)	Conditional UI rendering

---

4. Error / Edge Case Handling

Edge Case 1 — Rapid Favourite Toggling

Problem:
User taps the favourite icon very quickly.

Solution:
State updates should be handled in the **central state list**, ensuring consistent updates and avoiding duplicate state conflicts.

Edge Case 2 — LazyColumn Item Reuse

Problem:
LazyColumn recycles items when scrolling, which can reset UI state.

Solution:
Store state in the **data list or ViewModel**, not inside the composable item.

Edge Case 3 — Orientation Change

Problem:
Screen rotation may reset expanded items.

Solution:
Store expansion state using **ViewModel or rememberSaveable**.

Example:

var expanded by rememberSaveable { mutableStateOf(false) }

Test Cases

Test Case 1 — Favourite Toggle

Action	Expected Result
Tap heart icon	Icon becomes red and filled
Tap again	Icon returns to outlined

Test Case 2 — Scroll Behaviour

Action	Expected Result
Mark item as favourite	Scroll down and back up
Tap again	Favourite state remains unchanged

---

5. Reflection

Adding interactivity to list items increases complexity because the UI must manage dynamic state changes such as favourites and expansion. Developers must ensure that the state persists correctly during recomposition and scrolling. Rich interactive features improve user experience but also require careful state management and performance optimization. Balancing simplicity and functionality is important to keep the interface responsive and easy to maintain.